

PYTHON DATA ANALYTICS NOTES: CODES & CONCEPTS

Topics Covered: Interquartile Range (IQR) & Shape of Data (Symmetry & Skewness)

1. Interquartile Range (IQR) - Odd Number of Elements

Jab hamare paas odd count data points hote hain, tab exact central number median (Q2) ban jata hai. Manual logic mein hum median ko lower aur upper halves se drop kar dete hain aur phir un halves ka center nikalte hain.

```
# 9 Customers ka sorted gold purchase data (Odd count)
gold_purchases_odd = [10, 12, 15, 18, 20, 22, 25, 30, 35]
n = len(gold_purchases_odd)

# Step 1: Overall Median (Q2) Index
mid = n // 2 # 9 // 2 = 4 (Index 4 par 20 hai)

# Step 2: Split Lower Half & Upper Half (Drop Median)
lower_half = gold_purchases_odd[:mid] # [10, 12, 15, 18]
upper_half = gold_purchases_odd[mid+1:] # [22, 25, 30, 35]

# Step 3: Q1 & Q3 Calculation (Even count in halves, take average)
m_low = len(lower_half) # 4
q1 = (lower_half[m_low//2 - 1] + lower_half[m_low//2]) / 2 # (12 + 15) / 2 = 13.5

m_up = len(upper_half) # 4
q3 = (upper_half[m_up//2 - 1] + upper_half[m_up//2]) / 2 # (25 + 30) / 2 = 27.5

# Step 4: IQR
iqr = q3 - q1
print("Q1:", q1, "| Q3:", q3, "| IQR:", iqr)
```

Output: Q1: 13.5 | Q3: 27.5 | IQR: 14.0

2. Interquartile Range (IQR) - Even Number of Elements

Even elements mein koi single central element nahi hota. Isliye data splitting ke waqt koi bhi element drop nahi kiya jata, pehle aadhe elements lower half mein aur baaki aadhe upper half mein chale jaate hain.

```
# 10 Customers ka data (Even count)
gold_purchases_even = [10, 12, 15, 18, 20, 22, 25, 30, 35, 40]
n = len(gold_purchases_even)

# Step 1: Overall Median (Q2)
mid1 = n // 2 - 1
mid2 = n // 2
q2 = (gold_purchases_even[mid1] + gold_purchases_even[mid2]) / 2 # (20+22)/2 = 21.0

# Step 2: Split Data Without Dropping Anything
lower_half = gold_purchases_even[:n//2] # [10, 12, 15, 18, 20]
upper_half = gold_purchases_even[n//2:] # [22, 25, 30, 35, 40]

# Step 3: Q1 & Q3 (Odd count in halves, take direct center)
q1 = lower_half[len(lower_half)//2] # Index 2 par 15 hai
q3 = upper_half[len(upper_half)//2] # Index 2 par 30 hai

iqr = q3 - q1
print("Q1:", q1, "| Q3:", q3, "| IQR:", iqr)
```

Output: Q1: 15.0 | Q3: 30.0 | IQR: 15.0

3. IQR Direct Library Short Codes

Bina kisi loops ya slice extraction ke single statement se IQR nikalne ke liye NumPy aur SciPy direct tools provide karte hain. 100% manual match ke liye 'weibull' method use kiya jata hai.

NumPy Method:

```
import numpy as np
gold_purchases = [10, 12, 15, 18, 20, 22, 25, 30, 35, 40]

q1 = np.percentile(gold_purchases, 25, method='weibull')
q3 = np.percentile(gold_purchases, 75, method='weibull')
print("NumPy Direct IQR:", q3 - q1)
```

Output: NumPy Direct IQR: 15.0

SciPy Method (One-Liner):

```
from scipy import stats
gold_purchases = [10, 12, 15, 18, 20, 22, 25, 30, 35, 40]

print("SciPy Direct IQR:", stats.iqr(gold_purchases, interpolation='weibull'))
```

Output: SciPy Direct IQR: 15.0

4. Core Logic Breakdown: Frequency Counter Dictionary

Manual Mode calculation samajhne se pehle frequency table banana zaroori hai. Yeh core Python loop dictionary data structure ka use karke items ki ginti karta hai.

```
frequency_dict = {}
for item in gold_sales:
    if item in frequency_dict:
        frequency_dict[item] += 1
    else:
        frequency_dict[item] = 1
```

Line-by-Line Meaning:

- **if item in frequency_dict:** Agar koi number register mein pehle se likha hai, toh uski ginti purani value mein +1 kar do.
- **else:** Agar number pehli baar dikha hai, toh naya entry bana kar uske samne default value 1 likh do.

5. Core Logic Breakdown: Max Count Loop (Winner Selection)

Ginti ki table se us key (number) ko extract karna jiski value (count) sabse badi ho. Yeh logic competition style mein winner select karta hai.

```
max_count = 0
manual_mode = None
for number, count in frequency_dict.items():
    if count > max_count:
        max_count = count
        manual_mode = number
```

Competition Architecture:

Hum register ke har number aur uske count ko baari-baari uthate hain. Agar current number ka count hamare purane max_count se bada hai, toh record toot jata hai aur max_count badal kar naya count banta hai, aur manual_mode naya winner number ban jata hai.

6. Shape of Data - Symmetry Analysis

Symmetric data mein Distribution bilkul barabar hota hai. Iska mathematical criteria hota hai ki Mean, Median aur Mode teeno barabar hone chahiye.

Pure Manual Code:

```
gold_sales = [10, 20, 30, 20, 30, 10, 20]
gold_sales.sort() # [10, 10, 20, 20, 20, 30, 30]

# Manual Mean
manual_mean = sum(gold_sales) / len(gold_sales) # 20.0
# Manual Median
manual_median = gold_sales[len(gold_sales)//2] # 20

# Manual Mode Logic
frequency_dict = {}
for item in gold_sales: frequency_dict[item] = frequency_dict.get(item, 0) + 1
max_c = 0; manual_mode = None
for num, c in frequency_dict.items():
    if c > max_c: max_c = c; manual_mode = num

if manual_mean == manual_median == manual_mode:
    print("Data perfectly SYMMETRIC hai!")
```

Output: Data perfectly SYMMETRIC hai!

Short Import Method:

```
import statistics
gold_sales = [10, 20, 30, 20, 30, 10, 20]
if statistics.mean(gold_sales) == statistics.median(gold_sales) ==
statistics.mode(gold_sales):
    print("Symmetric via Library")
```

7. Shape of Data - Positive Skewness (Right-Skewed)

Business Context: Jab koi bada customer achanak high volume (Outlier) buy karta hai toh Mean aage khinch jata hai. Condition: Mean > Median.

Pure Manual Code:

```
gold_sales_positive = [10, 15, 12, 15, 10, 15, 80] # 80 is outlier
gold_sales_positive.sort()

mean_p = sum(gold_sales_positive) / len(gold_sales_positive) # 22.43
med_p = gold_sales_positive[len(gold_sales_positive)//2] # 15

if mean_p > med_p:
    print("Data POSITIVELY SKEWED (Right-Skewed) hai!")
```

Output: Data POSITIVELY SKEWED (Right-Skewed) hai!

Short Method:

```
import statistics
gold_sales_positive = [10, 15, 12, 15, 10, 15, 80]
if statistics.mean(gold_sales_positive) > statistics.median(gold_sales_positive):
    print("Result: Positively Skewed! 👍")
```

8. Shape of Data - Negative Skewness (Left-Skewed)

Business Context: Zyadatar orders bade hain par ek customer ne bahut chhota order kiya jisse Mean piche khinch gaya. Condition: $\text{Mean} < \text{Median}$.

Pure Manual Code:

```
gold_sales_negative = [75, 80, 78, 80, 72, 80, 5] # 5 is outlier
gold_sales_negative.sort()

mean_n = sum(gold_sales_negative) / len(gold_sales_negative) # 67.14
med_n = gold_sales_negative[len(gold_sales_negative)//2] # 78

if mean_n < med_n:
    print("Data NEGATIVELY SKEWED (Left-Skewed) hai! 🙌")
```

Output: Data NEGATIVELY SKEWED (Left-Skewed) hai!

Short Method:

```
import statistics
gold_sales_negative = [75, 80, 78, 80, 72, 80, 5]
if statistics.mean(gold_sales_negative) < statistics.median(gold_sales_negative):
    print("Result: Negatively Skewed! 🙌")
```

Master Conclusion: IQR data bikhraav ko bina outliers se prabhavit huye measure karta hai, jabki Symmetry aur Skewness hamein yeh batati hain ki data outliers ke karan kis direction mein lean ho raha hai.